EXAM CHEATSHEET · STUDY GUIDE · ONE-NIGHT REVISION PACK

CMS 702 — Data Structures & Computer Algorithms

PGD Computer Science, Rivers State University, Nkpolu-Oroworukwo, Port Harcourt · built from the lecturer's lecture notes, the official course outline and the solved past paper

Prepared by **Mbosinwa Awunor** · www.mbosinwa.dev

Exam: Monday 03 Aug 2026 **Time:** 11:00 – 14:00 **Venue:** the exam hall **Lecturer:** the lecturer **Units:** 3

HOW THIS EXAMINER MARKS — READ BEFORE ANYTHING ELSE

All five questions on the past paper were **bookwork**: definitions, numbered lists, one example per term, one diagram. Answer in that exact shape every time.

1. **Define first, always.** Even when the question only says "differentiate", open with a one-line definition of each term.
2. **"With examples" is a mark scheme, not decoration.** Never give a definition alone when the question says "with examples".
3. **Comparison question → table** with a "Basis of comparison" column. Faster to write, easier to mark.
4. **Open-ended lists → write 8–9 points, not 3.** Marks are awarded per valid point.
5. **Draw the diagram.** Q1 asked for it explicitly. Label the axes and mark n_0.

Confirmed past-paper questions: Q1 asymptotic analysis + 3 notations with diagram · Q2 define sort algorithm, search algorithm, dynamic programming with examples · Q3 hashing, hash function, hash table · Q4 importance of data structures · Q5 queue vs static data structure. **Highest-risk unseen topic:** BSTs — the lecturer spent the most board time there.

1 Definitions to write word-for-word

TERM	DEFINITION
Algorithm	A step-by-step procedure — a set of commands or instructions — to solve a specific problem.
Asymptotic analysis	Computing the running time of any piece of code or operation in a mathematical unit of computation, expressed as a function `f(n)`; the method of describing the **limiting behaviour** of an algorithm as input size grows.
Big O (O)	The **upper bound** of the growth rate of a function; measures worst-case performance — the algorithm will never be slower than this.
Theta (Θ)	The **tight bound** — expresses *both* the upper and the lower bound of the running time. The most precise notation.
Omega (Ω)	The **lower bound** — expresses only the best-case running time; the algorithm will never be faster than this.
Data structure	A data organization, management and storage format that enables efficient access and modification; a collection of data values, the relationships among them, and the operations applicable to them.
Hashing	Lookup — the most widely used technique to find aggregate data by *key* or *id*; mapping a large set of arbitrary data to a tabular index using a hash function. Stored in a hash map / hash table.
Hash function	The function that receives the input key and returns the index of an element in an array called the hash table.
Hash table	A data structure that maps keys to values using the hash function, storing data in an associative manner in an array where each data value has its own unique index.
Collision	Occurs when `h(x) = h(y)` — two different keys map to the same hash value.
Load factor	Number of items the hash table contains ÷ size of the hash table.

TERM	DEFINITION
Stack	An abstract data type that implements **LIFO**; open at one end only; insertion = **PUSH**, removal = **POP**.
Queue	An abstract data structure open at **both ends**; follows **FIFO**; insertion = **Enqueue()**, removal = **Dequeue()**.
Tree	A hierarchical, **non-linear** data structure consisting of nodes connected by edges.
Node	An entity that contains a key or value plus pointers to its child nodes.
BST	A binary tree in which the value of the left node is less than its parent and the value of the right node is greater than its parent.
Dynamic programming	A method of solving a complex problem by breaking it down into smaller units or sub-problems, solving each once and storing the result for reuse.
Graph	`G = (V, E)` — a non-linear data structure consisting of a set of vertices `V` and a set of edges `E` connecting pairs of vertices.
Recursive algorithm	A method that breaks a problem into smaller sub-problems and calls itself repeatedly until it reaches a base case it can solve directly.

2 The numbered lists most likely to be asked

Types of algorithms (10): Brute force · Recursive · Encryption · Backtracking · Search · Sort · Divide and conquer · Greedy · Dynamic programming · Randomized

Three cases in asymptotic analysis: Worst · Best · Average

Three asymptotic notations: Big O (upper) · Θ (tight) · Ω (lower)

Five complexity classes: O(1) constant · O(log n) logarithmic · O(n) linear · O(n^2) quadratic · O(2^n) exponential

Five sort algorithms from class: Merge · Quick · Bucket · Heap · Counting

Three components of hashing: Key · Hash function · Hash table

Four properties of a good hash function: efficiently computable · uniformly distributes the keys · minimizes collision · low load factor

Two collision-handling methods: Separate chaining (each cell points to a linked list of records) · Open addressing (all elements stored in the table itself)

Three types of data structure: Inbuilt/primitive (integer, float, boolean) · Derived (stack, queue, list, array) · Complex (linked list, tree, graph)

Six basic operations: Traversal · Searching · Sorting · Merging · Insertion · Deletion

Three types of linked list: Simple (singly) · Complex (doubly) · Circular

Three properties of a tree: one root node, which has no parent · each node has one parent only but may have many children · each node connects to its children via an edge

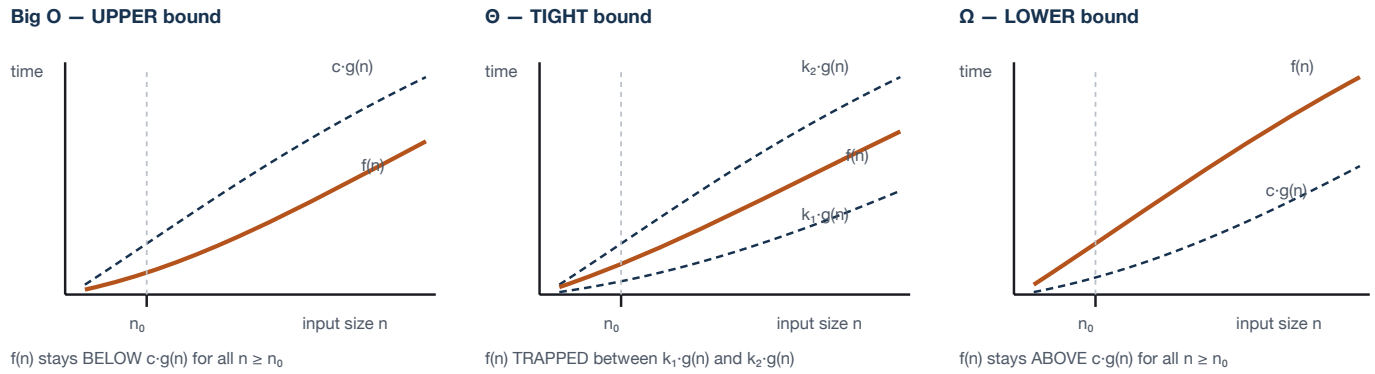
Four properties of a BST: left sub-tree strictly less · right sub-tree strictly greater · recursively true of every sub-tree · no duplicate keys

3 Asymptotic analysis & complexity PAST Q1

Why we use it: the actual running time of an algorithm depends on the hardware, the compiler and the language. By ignoring machine-dependent constants and concentrating on the **rate of growth**, asymptotic analysis gives a measure of efficiency that holds for any machine.

NOTATION	NAME	BOUND	CASE MEASURED	FORMAL DEFINITION
O	Big O	**Upper** bound	Worst case	$f(n) = O(g(n))$ if $\exists\ c,\ n_0 > 0$ such that $0 \le f(n) \le c \cdot g(n)$ for all $n \ge n_0$
Θ	Theta	**Tight** (upper *and* lower)	Average / exact	$f(n) = \Theta(g(n))$ if $0 \le k_1 \cdot g(n) \le f(n) \le k_2 \cdot g(n)$ for all $n \ge n_0$
Ω	Omega	**Lower** bound	Best case	$f(n) = \Omega(g(n))$ if $0 \le c \cdot g(n) \le f(n)$ for all $n \ge n_0$

The diagram the examiner asks for — draw all three

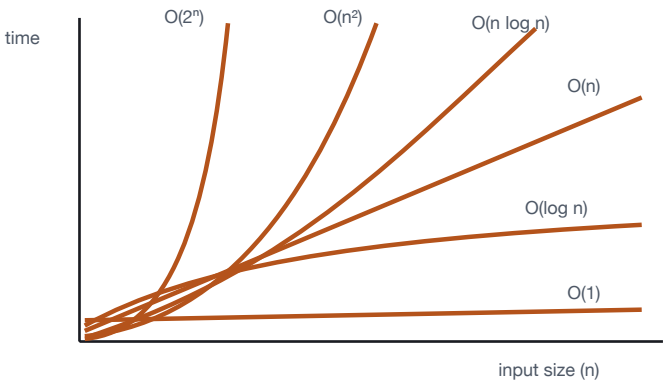


Vertical axis = running time, horizontal axis = input size n. Mark n_0 on every sketch — the bound only has to hold for $n \ge n_0$.

Complexity classes and the growth-rate curve

CLASS	BEHAVIOUR	EXAMPLE
O(1) constant	Fixed time regardless of data volume	Array index access
O(log n) logarithmic	Halves the problem size at each step	Binary search
O(n) linear	Time directly proportional to input size	Single loop / linear search
O(n log n)	Divide, solve, combine	Merge sort, heap sort
O(n²) quadratic	Proportional to the square of input size	Nested loops, bubble sort
O(2ⁿ) exponential	Grows rapidly with input size	Naïve recursive Fibonacci

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n)$$



Growth-rate curves drawn in class. The steeper the curve, the worse the algorithm scales.

Upper complexity bound guarantees performance in the worst case — the algorithm will never behave unexpectedly poorly. **Average complexity** is the expected performance given that all inputs of a certain size follow a certain distribution.

4 Sorting & searching PAST Q2

Sort algorithm — an algorithm that aims at arranging the elements of a list in a specific order: ascending or descending numerical order, or lexicographic (alphabetical) order. Sorting matters because a sorted collection

makes other operations — above all searching — far more efficient.

```
Unsorted Array:  9   1   3   2   7   4
                     ↓  sort
Sorted Array:    1   2   3   4   7   9
```

ALGORITHM	HOW IT WORKS	BEST	AVERAGE	WORST	SPACE	STABLE
Merge ★	Divide the list in half, sort each half, then merge the two sorted halves (divide & conquer)	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes
Quick ★	Pick a pivot, partition the list around it, recurse on both sides	O(n log n)	O(n log n)	$O(n^2)$	O(log n)	No
Bucket ★	Scatter elements into buckets, sort each bucket, concatenate	O(n+k)	O(n+k)	$O(n^2)$	O(n)	Yes
Heap ★	Build a max-heap, then repeatedly extract the maximum	O(n log n)	O(n log n)	O(n log n)	O(1)	No
Counting ★	Count occurrences of each key, then rebuild the list in order	O(n+k)	O(n+k)	O(n+k)	O(k)	Yes
Bubble	Repeatedly swap adjacent out-of-order pairs	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes
Selection	Repeatedly pick the minimum and place it at the front	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	No
Insertion	Insert each element into its place in the sorted prefix	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes

★ = the five named in the lecturer's notes. **Quick sort's $O(n^2)$ worst case** occurs when the pivot is always the smallest or largest element (e.g. already-sorted input with a first-element pivot). **Counting and bucket sort are not comparison sorts** — that is how they beat the O(n log n) comparison lower bound.

Search algorithms

A **search algorithm** is designed to find a specific target within a dataset, enabling effective retrieval of information — it answers "is this item present, and if so, where?"

BASIS	SEQUENTIAL (LINEAR)	BINARY
Requires sorted data	No	**Yes**
Method	Check each element from first to last	Compare with the middle, discard half, repeat
Best case	O(1)	O(1)
Worst case	**O(n)**	**O(log n)**
Works on	Arrays, linked lists	Arrays (needs random access)

Third example worth naming: BST search — exploits the BST property to discard half the tree at each node, O(log n) on a balanced tree.

BINARY SEARCH WORKED — FIND 7 IN `1 2 3 4 7 9`

```
Step 1:  1  2  3 [4] 7  9    middle = 4, 7 > 4
                             → discard the LEFT half
Step 2:              7 [9]   middle = 9, 7 < 9
                             → discard the RIGHT half
Step 3:             [7]      FOUND
```

Each step halves the search space — `n → n/2 → n/4 → …` — which is exactly why the cost is **$\log_2 n$**.

Dynamic programming PAST Q2(C)

Dynamic programming (DP) is a method of solving a complex problem by breaking it down into smaller units or sub-problems, solving each sub-problem **once**, and **storing its result** so it is not recomputed when the same sub-problem arises again.

It applies to problems with two features: **(1) overlapping sub-problems** — the same sub-problem is solved repeatedly; **(2) optimal substructure** — the optimal solution of the whole is built from optimal solutions of its parts.

DP vs divide-and-conquer: divide-and-conquer solves *independent* sub-problems; dynamic programming *stores and reuses* answers to *overlapping* ones. This is the time–space tradeoff — DP spends memory to save time.

EXAMPLE — FIBONACCI

```
Naïve recursion — O(2ⁿ)
fib(5) → fib(4) + fib(3)
         fib(3)+fib(2)  fib(2)+fib(1)
         ↑ fib(3), fib(2) recomputed
```

WITH DP — O(N)

```
fib[0] = 0
fib[1] = 1
for i = 2 to n:
    fib[i] = fib[i−1] + fib[i−2]
```

Other standard examples: the knapsack problem · longest common subsequence · matrix chain multiplication · Floyd–Warshall shortest paths.

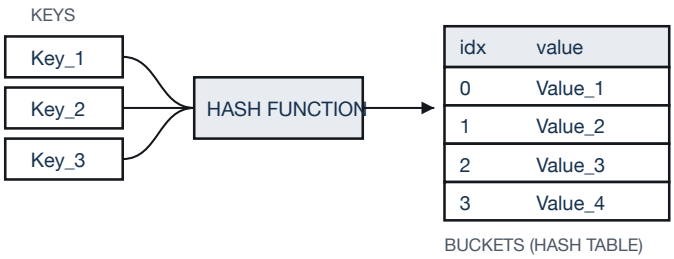
5 Hashing PAST Q3

Hashing means **lookup**. It is the most widely used technique to find aggregate data by key or id, and can also be described as mapping a large set of arbitrary data to a tabular index using a hash function. It is a method of representing dictionaries for large datasets; its central advantage is that **lookup, update and retrieval occur in constant time — O(1)** — instead of the O(n) needed to scan a list. The value obtained from the hash function is the **hash code**.

The three components

COMPONENT	DEFINITION TO WRITE
1. Key	Any string or integer used as the **input** to the hash function. It is the technique that determines the index or location for storing an item in a data structure.
2. Hash function	The function that **receives the input key and returns the index** of an element in an array called the hash table. It performs the transformation from key to location.
3. Hash table	A data structure that **maps keys to values** using the hash function, storing the data in an **associative manner** in an array where each data value has its own unique index.

DIAGRAM — KEY → HASH FUNCTION → BUCKET



WORKED EXAMPLE FROM CLASS

Store `{"ab", "cd", "efg"}` in a table of size 7, with `a=1, b=2 … g=7`. Rule: `index = sum mod table_size`.

```
ab  = 1 + 2     =  3  →  3 mod 7 = index 3
cd  = 3 + 4     =  7  →  7 mod 7 = index 0
efg = 5 + 6 + 7 = 18  → 18 mod 7 = index 4

index:  0     1    2    3     4    5    6
       | cd |    |    | ab | efg |    |    |
```

Always show three steps: the sum → the modulo → the slot. If two keys land on the same slot, name it a **collision** and give the fix.

GOOD HASH FUNCTION

A hash function that maps every item into its own unique slot is a **perfect hash function**. A good one should:

1. Be efficiently computable
2. Uniformly distribute the keys
3. Minimize collision
4. Have a low load factor

COLLISION

Occurs when `h(x) = h(y)` — two different keys map to the same hash value. Handled by:

- **Separate chaining** — each cell of the table points to a linked list of records
- **Open addressing** — all elements are stored in the hash table itself

LOAD FACTOR

`λ = items in table ÷ table size`

A table of size 10 holding 7 items has λ = **0.7**. A high load factor means more collisions and slower lookup — which is why keeping λ low is a property of a good hash function.

6 Trees — terminology, height and depth

A **tree** is a hierarchical, **non-linear** data structure representing **nodes connected by edges**. It is used as an abstract data type for data storage, and in data science for building predictive models because it handles large amounts of data well.

TERM	MEANING
Root	The node at the top of the tree; only one per tree; has no parent
Parent	Any node except the root has one edge upward to a node called its parent
Child	The node below a given node, connected by its edge downward
Leaf	A node with no child node (external node); height 0
Internal node	A node that has at least one child
Edge	The link between any two nodes
Path	The sequence of nodes along the edges of the tree
Sub-tree	The descendants of a node
Traversing	Passing through the nodes in a specific order
Levels	The generation of a node: root at level 0, its child at level 1, grandchild level 2 …
Keys	The value of a node, on which a search operation is carried out
Forest	A collection of disjoint trees

HEIGHT · DEPTH · DEGREE — THE CLASSIC 5-MARK QUESTION

- **Depth of a node** = number of edges from the **root down to** that node.
- **Height of a node** = number of edges from that node **down to its deepest leaf**.
- **Height of the tree** = height of the root = the longest path in edges to a leaf.
- **Degree of a node** = the total number of branches at that node.

> **Rule of thumb: depth counts downward from the root; height counts upward from the leaves.** Swapping them is the single most common lost mark.

```
          10        d=0, h=3   ← height of tree = 3
         /  \
        5    8      5: d=1, h=2
       / \  / \
      2   3 7  9    2: d=2, h=1
     /
    1               1: d=3, h=0  (leaf)
```

Types of tree — one line each

TYPE	DEFINITION	NOTE / FORMULA
General tree	No restriction on the number of children a node may have	e.g. a family tree, a folder structure
Binary tree	Every node has **at most two** children — left and right	Parent of the three types below
Full binary tree	Every node has either **0 or 2** children — never exactly one	No node with a single child
Perfect binary tree	Every internal node has exactly 2 children **and** all leaves are at the same level	$l = 2^h \cdot n = 2^{(h+1)} - 1$ h = 3 → 8 leaves, 15 nodes
Complete binary tree	Every level completely filled, leaves lean towards the **left**, and the last leaf may lack a right sibling	The shape used by a heap

7 Binary search trees — the highest-yield section

A **BST** is a node-based binary tree used to store and manage data in a **sorted** manner. Its advantage: it bridges the gap between the fast lookup of a sorted array and the flexible modification of a linked list.

The four properties — every node must strictly obey them: **(1)** every node in the **left** sub-tree holds a value **strictly less** than the parent's; **(2)** every node in the **right** sub-tree holds a value **strictly greater** than the parent's; **(3)** the rule applies **recursively** — every sub-tree is itself a valid BST; **(4) no duplicate keys**.

The three traversals — do not mix these up

TRAVERSAL	ORDER	SHORTHAND
In-order	Left → **Root** → Right	L-Root-R
Pre-order	**Root** → Left → Right	Root-L-R
Post-order	Left → Right → **Root**	L-R-Root

Memory hook: the position of *Root* in the name is its position in the visit order — **pre** = first, **in** = middle, **post** = last. Left always comes before Right.

> **Key exam fact:** the in-order traversal of a BST always comes out **sorted ascending**. Use it as a free sanity-check on any tree you draw.

```
          20
         /  \
        8    22
       / \
      4   12
         /  \
        10   14

In-order   : 4, 8, 10, 12, 14, 20, 22   ← sorted
Pre-order  : 20, 8, 4, 12, 10, 14, 22
Post-order : 4, 10, 14, 12, 8, 22, 20
```

In-order successor & predecessor WORKED TWICE IN CLASS

- **In-order successor of K** = the node with the **smallest value greater than** K. If no such node exists, return **−1**.
- **In-order predecessor of K** = the **previous node in the in-order traversal**. It is **null** for the first node.

> **Fastest method under exam pressure:** write out the full in-order traversal, then simply read off the neighbour to the right (successor) or left (predecessor).

On the tree above, in-order = `4, 8, 10, 12, 14, 20, 22`:

K	SUCCESSOR	PREDECESSOR
4	8	**null** (smallest)
8	**10** ← class assignment	4
10	12	8
14	20	12
22	**−1** (largest)	20

Building a BST — the two exam formats

(A) FROM A LIST OF VALUES

Insert one value at a time starting from the root: go **left if smaller**, **right if larger**. From `(37, 21, 13, 40, 36, 50)`:

```
          37
         /  \
       21    40
      /     /  \
    13    36    50

In-order   : 13, 21, 36, 37, 40, 50   ← sorted ✓
Pre-order  : 37, 21, 13, 40, 36, 50
Post-order : 13, 21, 36, 50, 40, 37
```

(B) FROM TWO TRAVERSALS

Pre-order + In-order: the **first** element of the pre-order is the root → find it in the in-order → everything to its left is the left sub-tree, everything to its right is the right sub-tree → recurse on each side.

Post-order + In-order: identical method, except the **last** element of the post-order is the root.

```
Pre-order = 1, 2, 4, 8, 9, 10, 11, 5, 3, 6, 7
In-order  = 8, 4, 10, 9, 11, 2, 5, 1, 6, 3, 7

Root = 1 → left = 8,4,10,9,11,2,5 | right = 6,3,7
```

```
              1
            /   \
           2     3
          / \   / \
         4   5 6   7
        / \
       8   9
          /  \
         10   11
```

8 Linear structures — arrays, linked lists, stacks, queues PAST Q5

REPRESENTATION IN MEMORY

```
ARRAY
Name: Int Array(10)
        ↑       ↑
      Type    Size
Elements: {35,33,42,10,14,19,27,44,26,31}

LINKED LIST
[Head] → [Data|•] → [Data|•] → [Data|•] → NULL
           Node       Node       Node
```

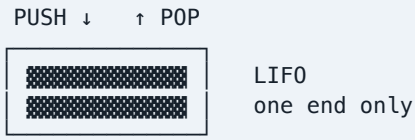
Types of array: fixed-size — cannot be altered, indexes are numbered; **dynamic-size** — can be altered/resized, may be one-, two- or three-dimensional.
Types of linked list: simple (singly) · complex (doubly) · circular.

DELETING A NODE FROM A SINGLY LINKED LIST

```
Head → [A|•] → [TARGET|•] → [C|•] → NULL
          └──────────────────────↗
Locate the target, then redirect the previous
node's pointer past it to the target's next node.
```

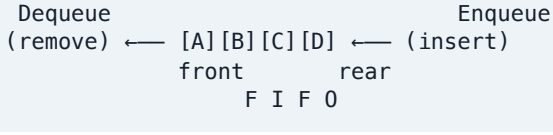
STACK — LIFO

An abstract data type named after a real-world stack (a pile of plates, a stack of pizza), which allows operations at **one end only**, one at a time. Insertion is **PUSH**, removal is **POP**. It can be implemented with arrays, pointers, linked lists or structures, and can be fixed-size or dynamic.



QUEUE — FIFO

Similar to the stack but **open at both ends** and following **FIFO**. **Enqueue()** adds at the rear, **Dequeue()** removes from the front.



Stack vs Queue

BASIS	STACK	QUEUE
Principle	**LIFO** — Last In, First Out	**FIFO** — First In, First Out
Open at	One end only	Both ends
Operations	PUSH (insert), POP (remove)	Enqueue() (insert), Dequeue() (remove)
Pointers used	One — *top*	Two — *front* and *rear*
Analogy	A pile of plates, a stack of pizza	A queue of people at a counter
Used in	Recursion & function calls, undo, **DFS**	CPU scheduling, printer spooling, **BFS**

Queue vs static data structure — the exam trap PAST Q5

The comparison is not symmetrical, and saying so earns marks. A **queue** names a specific **ADT defined by its behaviour** (FIFO); "**static**" names a whole **memory-allocation category** whose standard example is the fixed-size array. State that distinction first, then compare the queue against the fixed-size array.

#	BASIS	QUEUE	STATIC DATA STRUCTURE (FIXED-SIZE ARRAY)
1	What it is	An abstract data type, defined by *behaviour* (FIFO), not by storage	A storage category, defined by *how memory is allocated* — fixed at compile time
2	Size	Logically unbounded; grows and shrinks at run time (linked implementation)	Fixed and declared in advance; cannot grow or shrink at run time
3	Memory allocation	Dynamic, at run time (heap)	Static, at compile time (stack), allocated in advance
4	Access pattern	Restricted — insert at the **rear**, remove from the **front** only	Random access — any element reached directly by index `A[i]` in O(1)

#	BASIS	QUEUE	STATIC DATA STRUCTURE (FIXED-SIZE ARRAY)
5	Order of operation	Strictly FIFO	No ordering rule; any order the programmer chooses
6	Points of entry/exit	Open at **both ends**	Every position equally accessible; "ends" does not apply
7	Operations	Enqueue() and Dequeue() only	Insert, delete, traverse, search, update at any index — but capacity is fixed
8	Memory efficiency	Uses exactly as much memory as it holds	May waste memory if under-filled, or overflow if declared too small
9	Insert/delete cost	O(1) at the designated end	O(n) in the middle — elements must be shifted
10	Examples	Printer queue, CPU scheduling, BFS, call-centre line, keyboard buffer	Fixed-size array, a record/struct, a fixed-size matrix

Bonus sentence worth a mark: a queue can itself be implemented on top of a static array (a circular or bounded queue), inheriting a fixed capacity and the possibility of overflow — the FIFO *behaviour* is what makes it a queue; the array is only the storage underneath.

9 Graphs and their representation — OUTLINE TOPIC — NOT IN THE NOTES

A **graph** `G = (V, E)` is a non-linear data structure consisting of a set of **vertices** (nodes) `V` and a set of **edges** `E` connecting pairs of vertices.

Types: undirected — edges have no direction, (A,B) = (B,A) · **directed (digraph)** — edges have direction, A→B ≠ B→A · **weighted** — each edge carries a cost/weight · **cyclic / acyclic** — contains a cycle or not. *A tree is a connected acyclic graph.*

BASIS	ADJACENCY MATRIX	ADJACENCY LIST
Structure	V×V matrix, `M[i][j] = 1` if an edge i→j exists	Array of V lists; each list holds a vertex's neighbours
Space	O(V²)	O(V + E)
Check edge (u,v)	**O(1)**	O(degree of u)
List all neighbours	O(V)	O(degree)
Best for	**Dense** graphs	**Sparse** graphs

EXAMPLE — UNDIRECTED, VERTICES A B C, EDGES AB AND BC

```
Adjacency Matrix          Adjacency List
    A  B  C               A → B
A [ 0  1  0 ]             B → A, C
B [ 1  0  1 ]             C → B
C [ 0  1  0 ]
```

Traversals — link them back to the notes:
BFS (Breadth-First Search) uses a **queue** (FIFO) and visits level by level — O(V + E).
DFS (Depth-First Search) uses a **stack** (LIFO) or recursion and goes as deep as possible first — O(V + E).

10 Memory, recursion & run-time storage management — OUTLINE TOPIC

Stack allocation vs heap allocation

BASIS	STACK	HEAP
What lives there	Local variables, parameters, return addresses	Dynamically allocated objects
Allocated by	Compiler, automatically on function call	Programmer at run time (`malloc`/`new`)

Recursive algorithms

A recursive algorithm solves a problem by calling itself on smaller sub-problems. Every recursion needs **(1) a base case** that stops it and **(2) a recursive case** that moves toward the base case.

```
factorial(n):
    if n <= 1:  return 1                  # base case
    else:  return n * factorial(n - 1)    # recursive
```

BASIS	STACK	HEAP
Freed by	Automatically on function return	Programmer (`free`/`delete`) or garbage collector
Size	Fixed, small	Large, grows at run time
Speed	Very fast (move the stack pointer)	Slower (search for a free block)
Order	LIFO	Any order
Failure mode	Stack overflow	Memory leak / fragmentation

Run-time storage management is the system's job of allocating memory while a program runs and reclaiming it afterwards — the stack for call frames, the heap for dynamic data, plus **garbage collection** (automatic reclamation of unreachable objects) or manual deallocation.

Don't confuse: the *stack* (memory region) with the *stack ADT* (LIFO structure), or the *heap* (memory region) with the *heap* data structure (the tree used by heap sort). Examiners like testing exactly this.

- Each call consumes a **stack frame** → recursion costs O(depth) space.
- A missing or wrong base case causes infinite recursion → **stack overflow**.
- **Recursion vs iteration:** recursion is shorter and natural for trees and divide-and-conquer; iteration avoids call-stack overhead.

Time–space tradeoff

An algorithm can often be made **faster by using more memory**, or made to **use less memory at the cost of running longer**. Examples to cite:

- A **hash table** spends O(n) extra space to turn an O(n) search into O(1) — space buys speed.
- **Merge sort** uses O(n) extra space to guarantee O(n log n); **quick sort** uses only O(log n) but risks $O(n^2)$.
- **Counting sort** uses an O(k) count array to avoid comparisons entirely.
- **Memoization** in DP stores sub-results to avoid recomputation.

STRINGS & STRING PROCESSING

A **string** is an array/sequence of characters, usually terminated by a null character `\0` in C-style implementations. Operations: length, concatenation, substring, comparison, pattern search, reverse. **Naïve pattern matching** is O(n·m); **KMP** improves it to O(n + m) using a pre-computed prefix table. Strings are the standard input to a hash function.

RECORDS / STRUCTURES

A **record** (struct) is a composite data type grouping fields of possibly **different** types under one name — e.g. a `Student` record with `name` (string), `matric_no` (int), `cgpa` (float). Contrast with an **array**, which holds many elements of the **same** type. A record is the building block of a **node**: data field + pointer field.

NUMERICAL ALGORITHMS

Algorithms operating on numeric data, where **precision and error** matter as much as speed: **Euclid's algorithm** (GCD) · **Newton–Raphson** (roots) · **fast exponentiation** by squaring, O(log n) · **Sieve of Eratosthenes**, O(n log log n) · **matrix multiplication**, naïve $O(n^3)$. Key point to mention: floating-point **round-off error** accumulates, so these algorithms are judged on **numerical stability**, not complexity alone.

11 Importance of data structures — write nine points PAST Q4

Open with the definition, then list. Marks are per valid point, so write eight or nine, not three.

1. **Efficient access and retrieval.** The right structure finds an item quickly: unsorted list O(n), sorted array with binary search O(log n), hash table O(1). The structure — not the processor speed — decides this.
2. **Efficient use of memory.** Structures determine how data is represented in memory: static structures such as arrays reserve memory in advance; dynamic structures such as linked lists allocate only as needed, avoiding waste.
3. **They make algorithms possible and efficient.** Every algorithm runs on some structure — binary search needs a sorted array, BFS needs a queue, DFS needs a stack, heap sort needs a heap. A good choice can cut an algorithm from $O(n^2)$ to O(n log n).
4. **They model real-world relationships.** Trees model hierarchies (file systems, org charts, family trees), graphs model networks (roads, social networks, the internet), queues model waiting lines, stacks model undo and function-call handling.

5. **Reusability and abstraction.** A structure defines an interface (the ADT) separately from its implementation, so the same stack or queue is reused across programs without rewriting.
6. **They support the six standard operations** — traversal, searching, sorting, merging, insertion, deletion — in an organised and predictable way.
7. **Program maintainability and readability.** Properly structured data produces cleaner, shorter code that is easier to debug and extend.
8. **They let the time–space tradeoff be managed deliberately.** A hash table spends memory to make search constant-time; a linked list spends memory on pointers to make insertion and deletion cheap.
9. **They are the foundation of larger systems.** Databases use B-trees and indexes, compilers use parse trees and symbol tables, operating systems use queues for scheduling, and AI uses graphs and trees for search.

12 Last-hour drill

Ten easy marks people lose

1. Writing **O** for the tight bound — the tight bound is **Θ**.
2. Saying a queue is open at one end — that is the **stack**. A queue is open at **both** ends.
3. Forgetting the **−1** when an in-order successor does not exist (and **null** for a missing predecessor).
4. Swapping **depth** and **height**. Depth counts down from the root; height counts up from the leaves.
5. Running binary search on **unsorted** data — it requires a sorted dataset. O(log n).
6. Forgetting that a BST's in-order traversal is **sorted** — the free check on every tree you draw.
7. Defining a term without giving the **example** the question asked for.
8. Answering a comparison question in prose instead of a **table**.
9. Drawing an asymptotic diagram with no **axes labels** and no n_0.
10. Confusing the **stack/heap memory regions** with the **stack/heap data structures**.

Self-test — cover the right column

PROMPT	ANSWER
Three asymptotic notations + what each bounds	O upper/worst · Θ tight/both · Ω lower/best
Order the five complexity classes	$O(1) < O(\log n) < O(n) < O(n^2) < O(2^n)$
Three components of hashing	Key · hash function · hash table
Collision definition + two fixes	$h(x)=h(y)$ · separate chaining · open addressing
Load factor of a size-10 table holding 7 items	0.7
Hash of "efg" in a 7-slot table	$5+6+7 = 18 \rightarrow 18 \bmod 7 =$ index 4
Perfect binary tree, h = 3: leaves and nodes	$l = 2^3 = 8$ leaves · $n = 2^4 - 1 = 15$ nodes
Depth and height of node 5 in the §6 tree	d = 1, h = 2
Traversal order of post-order	Left → Right → Root
BST from 37,21,13,40,36,50 — pre-order	37, 21, 13, 40, 36, 50
In-order successor of 8 in the §7 tree	10
Nested loop over n elements	$O(n^2)$ — n × n operations
Data structure each traversal uses	BFS → queue · DFS → stack
Merge sort space vs quick sort worst case	O(n) space · $O(n^2)$ worst
Three types of linked list	Simple (singly) · complex (doubly) · circular
Six basic operations on data structures	Traversal, searching, sorting, merging, insertion, deletion

TIMING PLAN FOR A 3-HOUR PAPER

Read the whole paper for 5 minutes and start with the question you know best — usually a definitions question, which banks marks fast. Budget roughly **30 minutes per question** for five questions, leaving 20 minutes to draw and label diagrams properly and 10 minutes to re-read. If a question has parts (a), (b), (c), **split the time evenly** and never let one part eat another's marks. Any question that says "with a diagram" gets the diagram **first**, before the prose — it is the part you will run out of time for.